https: / / github.com / fhlsteven / CLR_via_CSharp / blob / main / chapters / ch22_CLRHostingAndAppDomain.md 我现在在学习《CLR via C#》这本书的第22章，内容如链接所示

用户:

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch22_CLRHostingAndAppDomain.md

我现在在学习《CLR via C#》这本书的第22章，内容如链接所示，你总结一下当前章的各节内容，我看你的总结学习就行了，不看文章了。

元宝:

第22章《CLR 寄宿和 AppDomain》讲的是 **.NET 程序到底是怎么被"托管"起来的，以及在同一个进程里怎样做到隔离**——这是 ASP.NET、SQL Server 这类宿主程序的基石。全书主线到了第22章开始进入"核心机制"部分，这一章的概念在 .NET Framework 时代非常重要。

下面按原文章节顺序给你梳理。

22.1 CLR 寄宿（CLR Hosting）

核心问题：Windows 只认 PE 文件，不认托管代码。那 CLR 自己又是怎么"挂"进一个 Windows 进程里去的？

关键事实：

- CLR 被实现为一个 **COM 服务器（DLL）**，有标准 COM 接口和 GUID，安装 .NET Framework 时在 Windows 注册表里注册
- 任何 Windows 应用程序都可以寄宿（host）CLR，步骤大致是：
  1. Windows 检查 EXE 头，决定创建 32 位 / 64 位 / WoW64 进程
  2. 把对应版本的 `MSCorEE.dll`（垫片 shim）加载进进程
  3. 主线程调用 `CLRCreateInstance` → 拿到 `ICLRMetaHost` → `GetRuntime` 拿到 `ICLRRuntimeInfo` → `GetInterface` 拿到 `ICLRRuntimeHost`
- 通过 `ICLRRuntimeHost`，宿主可以做五件事：**设置宿主管理器、获取 CLR 管理器、初始化并启动 CLR、加载程序集并执行代码、停止 CLR**

几个必须记住的细节：

> 💡 CLR 一旦加载进进程，**永远不能卸载**——唯一能把它从进程里去掉的办法是终止整个进程

> 💡 .NET Framework 4 之前，一个进程只能加载一个版本的 CLR（v1.0 / v1.1 / v2.0 三选一）；.NET Framework 4 开始支持 v2.0 和 v4.0 **并存**于同一进程

宿主通过"宿主管理器"可以深度参与 CLR 的决策：**内存分配、线程调度与同步、程序集加载、垃圾回收启停通知、超时通知**；通过"CLR 管理器"可以**禁止某些类/成员被使用**，还能指定当 AppDomain 卸载、CLR 停止、堆栈溢出等事件发生时调用哪个方法

22.2 AppDomain

一句话定义：AppDomain 是**一组程序集的逻辑容器**，它的唯一作用就是**隔离**。

CLR COM 服务器初始化时会自动创建一个 AppDomain，称为**默认 AppDomain**——它只有在 Windows 进程终止时才会被销毁。宿主也可以通过非托管 COM 接口或托管 API 要求 CLR 创建额外的 AppDomain。

AppDomain 提供的四大隔离能力：

能力	说明
代码隔离	一个 AppDomain 中的代码不能直接访问另一个 AppDomain 创建的对象
可卸载	AppDomain 可以整体卸载（但不能只卸载其中某个程序集）
可保护	创建时可施加一个权限集
可配置	创建后关联一组配置：搜索路径、版本绑定重定向、卷影复制、加载器优化

关于 Loader 堆的重要细节（这是 Richter 讲得最透的部分）：

- 每个 AppDomain 有**自己的 Loader 堆**，记录自创建以来访问过的类型
- 即使两个 AppDomain 都加载了 `System.dll`，**类型对象的内存也不共享**，JIT 编译出的本机代码也各自关联
- 这种"看似浪费"的设计是为了保证：卸载某个 AppDomain 时不会影响其他 AppDomain
- 例外：`MSCorLib.dll`（含 `System.Object`、`System.Int32` 等核心类型）以 **AppDomain 中立**方式加载，所有 AppDomain 共享其类型对象，CLR 为它维护一个特殊的 Loader 堆

> ⚠️ 这是 .NET Framework 的概念。在 .NET Core / .NET 5+ 中，AppDomain 的隔离功能主要由 `AssemblyLoadContext` 替代

跨越 AppDomain 边界访问对象

既然 AppDomain 之间不能直接访问对象，那跨域调用怎么办？答案是两种"封送（marshaling）"语义：

1. 按引用封送（Marshal-by-Reference）

- 对象必须**派生自** `System.MarshalByRefObject`
- 通过 `CreateInstanceAndUnwrap` 把一个 AppDomain 中的对象按引用封送到另一个 AppDomain，返回的是**代理对象**
- 代价：跨边界调用性能较差

2. 按值封送（Marshal-by-Value）

- 对象**未派生自** `MarshalByRefObject` **但可序列化**时采用
- CLR 把对象序列化成字节数组，从源 AppDomain 复制到目标 AppDomain 后反序列化，并在目标 AppDomain 加载相应程序集、实例化出新对象（书里称为"精准复制"）

简单说：**想跨域传引用就用** `MarshalByRefObject`**，想跨域传副本就让对象可序列化。**

卸载 AppDomain

调用 `AppDomain.Unload` 静态方法即可，但这个方法**会阻塞调用线程**，CLR 在背后做了一整套动作：

1. **挂起**进程中所有曾执行过托管代码的线程
2. 检查所有线程栈，找出哪些线程正在被卸载的 AppDomain 中执行，或将来可能返回到该 AppDomain
3. 强制这些线程抛出 `ThreadAbortException`，让它们展开调用栈、执行 `finally` 块做清理
4. 对正在执行 `finally` 块、 `catch` 块、类构造函数、临界执行区或非托管代码的线程，**不立即中止**，等这些代码块执行完再抛异常
5. 所有相关线程离开 AppDomain 后，CLR 遍历堆，给所有指向该 AppDomain 对象的**代理对象打上失效标记**——之后再调用这些代理的方法会抛 `AppDomainUnloadedException`
6. 强制执行一次垃圾回收，调用待回收对象的 `Finalize` 方法
7. 恢复其余线程，`AppDomain.Unload` 的调用者继续运行

> ⚠️ 这里有个反常细节：普通未处理异常会让 CLR 杀掉进程，但 `ThreadAbortException` 如果不被捕获，**线程死掉而进程继续运行**

监视 AppDomain

CLR 提供了一套**应用程序域资源监视（ARM）**机制，宿主可以监控每个 AppDomain 的 CPU 时间和内存使用量。

启用方式很简单——设置静态属性：

```
AppDomain.MonitoringIsEnabled = true;
```

书上给出了一个工具类 `AppDomainMonitorDelta`（实现 `IDisposable`），它在静态构造函数里打开监视，然后用三个字段跟踪：

- `m_thisADCpu`：`TimeSpan` 类型，CPU 时间
- `m_thisADMemoryInUse`：当前使用的内存
- `m_thisADMemoryAllocated`：累计分配的内存

通过这个工具类可以在一段代码执行前后对比 AppDomain 的资源消耗增量。

本章核心要点回顾

把这几句话记住，这一章就算过关了：

1. **CLR 是个 COM DLL**，宿主通过 `ICLRRuntimeHost` 接口把它加载进自己的进程，且 **CLR 一旦加载就不能卸载，除非杀掉进程**
2. **AppDomain 的唯一目的是隔离**——代码隔离、可卸载、可保护、可配置
3. 跨 AppDomain 访问对象只有两条路：**按引用封送**（`MarshalByRefObject`）或**按值封送**（可序列化）
4. `AppDomain.Unload` 是个**同步阻塞调用**，CLR 通过挂起线程、抛 `ThreadAbortException`、GC 回收等一连串动作优雅卸载
5. 同一进程内多个 AppDomain **不共享类型对象和 JIT 代码**（`MSCorLib` 除外，它以 AppDomain 中立方式加载）

> 📌 学完这一章你应该建立起这样一个心智模型：**宿主进程 → CLR（COM 服务器）→ 一个或多个 AppDomain → 每个 AppDomain 里装着若干程序集**。下一章（第23章）就会顺着这条线讲"程序集加载和反射"，正好衔接。

需要我把某一节再展开讲讲吗？比如跨 AppDomain 封送的性能陷阱、或者 `AppDomain.Unload` 在生产环境中的坑，这两块是面试和实际开发里最容易踩雷的地方。

▾ 引用 13 篇资料作为参考

1. 🔗 《CLR via C#》读书笔记 之 CLR寄宿和AppDomain
2. 🔗 第 22 章 CLR寄宿和AppDomain
3. 🔗 GitHub - fhlsteven/CLR_via_CSharp: CLR Via C# 第四版
4. 🔗 CLR Via C# 读书笔记-第22章(CLR 寄宿和 AppDomain)
5. 🔗 应用程序域
6. 🔗 CLR Via CSharp读书笔记(22):CLR寄宿和应用程序域
7. 🔗 Using Application Domains - .NET Framework
8. 二十二、CLR寄宿与AppDomain(CLR Hosting and App Domains)
9. CLR第二十二章—CLR寄宿与AppDomain(CLR Hosting and App Domains)
10. 🔗 第22章 CLR寄宿和AppDomain
11. 🔗 CLR Hosted Environment
12. 🔗 appdomain
13. 🔗 重温CLR(十六) CLR寄宿和AppDomain

用户:

“22.1 CLR寄宿（CLR Hosting）”还是不太懂，看完迷迷糊糊，各种名词都不太懂。

元宝:

没问题，这一节之所以难懂，是因为 Richter 默认你已经熟悉 **COM** 和 **Windows 进程加载 DLL** 的那套机制。我用"零基础视角"重新讲一遍，把每个名词都翻译成人话。

先建立一个核心认知：CLR 本质上是个 DLL

你可能以为 ".NET 程序一运行，CLR 就自动出现了"——其实不是。

真相是：**CLR 只是一段 C++ 代码编译出来的 DLL 文件**（在 .NET Framework 里叫 `clr.dll`）。它自己**不会自己跑起来**，必须有一个 Windows 进程把它加载进去，它才能在那个进程里工作。

> 💡 "寄宿（Hosting）"这个词的意思就是：**让 CLR 寄居在某个 Windows 进程里**

谁来当这个"房东"？最常见的房东就是 `dotnet.exe` 或者 `xxx.vshost.exe`——也就是你运行 C# 程序时，Windows 真正启动的那个原生 EXE。但这个 EXE 本身是用 C/C++ 写的，**它不懂托管代码**，它只负责一件事：**把 CLR 这个 DLL 请进自己的进程空间。**

除了 `dotnet.exe`，**IIS（ASP.NET）、SQL Server、PowerShell** 等都是 CLR 的房东——它们把自己的进程"借"一块给 CLR 住着，这样就能在自己的进程里跑 C# 代码。这就是"寄宿"这个概念的存在意义。

加载 CLR 的完整链路（重点理解这 4 步）

Windows 启动一个托管程序时，发生的事其实是一条**接力赛**：

```
Windows 内核
   ↓ ① 创建进程，发现是 .NET 程序
   ↓    于是加载 MSCorEE.dll（垫片）
MSCorEE.dll（垫片 / Shim）
   ↓ ② 根据程序的目标框架版本，找到对应的 clr.dll
   ↓    调用 COM 接口把 CLR 请进来
CLR（clr.dll）
   ↓ ③ 创建默认 AppDomain
   ↓ ④ 加载入口程序集，JIT 开始编译 Main 方法
你的 Main 方法终于跑起来了
```

逐站解释：

① 垫片 `MSCorEE.dll` 是什么？

`MSCorEE` = **Microsoft Component Object Runtime Execution Engine**。它是 .NET Framework 安装时放到 `C:\Windows\System32\` 下的一个**很小的 DLL**。

为什么需要它？因为一个机器上可能装了**多个版本的 CLR**（v2.0、v4.0……）。Windows 加载 EXE 时还不知道这个程序到底需要哪个版本的 CLR，所以先统一加载 `MSCorEE.dll` 这个"中间人"，由它去判断："哦，这个程序是 .NET 4.8 的，那我去加载 v4.0 的 `clr.dll`"。

> 📌 你可以把 `MSCorEE.dll` 理解成**CLR 的"前台接待"**——你（EXE）来了先找它，它帮你转接到正确的 CLR 版本

② COM 接口是什么鬼？

COM（Component Object Model）是 Windows 上古时期就定下的一种"二进制接口标准"——它规定了：**一个 DLL 暴露出来的函数，别人怎么调用它。**

CLR 是以 COM 服务器的形式实现的，也就是说它向外暴露了一组标准的 COM 接口。房东进程通过这些接口"遥控"CLR。

Richter 书里提到的那串接口调用：

```
CLRCreateInstance()        // 拿到 IMetaHost 工厂
  → GetRuntime()           // 指定版本（如 "v4.0.30319"），拿到 ICLRRuntimeInfo
    → GetInterface()       // 拿到 ICLRRuntimeHost（这才是真正控制 CLR 的把手）
```

翻译成人话就是：

- `CLRCreateInstance`：跟系统说"我要找个 CLR"
- `GetRuntime("v4.0.30319")`：明确"我要 4.0 这个版本"
- `GetInterface(ICLRRuntimeHost)`：拿到**控制 CLR 的遥控器**

③ `ICLRRuntimeHost` 这个"遥控器"能干什么？

拿到这个接口后，宿主就能对 CLR 发号施令了，主要是 5 件事：

能力	通俗解释
设置宿主管理器	告诉 CLR："内存分配你别全权管，遇到 XX 情况要先问我"
获取 CLR 管理器	反过来，宿主也可以限制 CLR：比如"不允许加载某某程序集"
初始化并启动 CLR	按下开机键
加载程序集并执行	让 CLR 开始干活
停止 CLR	关机（但其实 CLR 不支持真正卸载，只能随进程死）

④ 为什么"CLR 不能卸载"是个大事？

这是个反直觉的点：DLL 不是可以 `FreeLibrary` 卸掉的吗？

CLR 特殊。因为它一旦启动，就会在进程里**创建一堆内部状态**（线程池、GC 堆、类型系统、JIT 缓存……）。这些状态深度交织，**微软觉得做"部分卸载"风险太大，干脆不支持。**

所以规则是：**CLR 加载后，除非整个进程退出，否则它永远在那。**这也是为什么：

- IIS 里你改了 web.config，想让 CLR 重新加载？不行，**必须重启整个 w3wp.exe 进程**
- 想在一个进程里切换 CLR 版本？.NET Framework 4 之前**完全不行**，4.0 开始才允许 v2.0 + v4.0 两个 CLR **并存**（注意是并存，不是替换）

宿主管理器和 CLR 管理器：谁说了算？

这是本节最容易懵的地方。简单说就是**双向控制**：



- **宿主管理器（Host Manager）**：宿主**主动设置**给 CLR 的一组"政策"。比如宿主可以说："内存紧张时先通知我"、"线程调度你要听我的"、"程序集加载要经过我批准"
- **CLR 管理器**：宿主通过它**反向限制 CLR**。比如："`System.IO.File` 的 `Delete` 方法不准用"、"AppDomain 卸载时要回调我的 XX 方法"

> 💡 为什么要这么复杂？因为像 SQL Server 这种宿主，**它本身就是个庞大的 C++ 程序**，CLR 只是它的一个"插件"。宿主必须能压制 CLR，否则 CLR 乱分配内存、乱开线程，SQL Server 就崩了

把整个流程串成一句话

当你双击一个 `.exe` 文件：

1. Windows 创建一个原生进程
2. 发现这是 .NET 程序，**加载 `MSCorEE.dll` 这个垫片**
3. 垫片根据程序版本，**找到对应的 `clr.dll` 并加载**
4. 宿主通过 COM 接口 `ICLRRuntimeHost`，**启动 CLR**
5. CLR **创建默认 AppDomain**，**加载你的程序集**，**JIT 编译 `Main` 方法**
6. 你的 C# 代码终于开始执行

这节你只需要记住 3 件事

> 🎯 **第一**：CLR 是个 DLL，不是什么魔法。它必须被某个 Windows 进程"请"进去才能工作，这个过程叫**寄宿**

> 🎯 **第二**：`MSCorEE.dll` 是垫片（前台），`clr.dll` 才是真正的 CLR（后台）。宿主通过 COM 接口 `ICLRRuntimeHost` 遥控 CLR

> 🎯 **第三**：CLR 加载后**不能卸载**，除非进程退出。这是 .NET Framework 时代很多架构决策的根本原因

如果你愿意，我可以接着把 **22.2 AppDomain** 也用这种"零基础视角"重讲一遍——特别是 **Loader 堆** 和 **跨域封送** 这两个点，用比喻讲会比书上好懂很多。要继续吗？

用户:

你说的“谁来当这个"房东"？最常见的房东就是 dotnet.exe 或者 xxx.vshost.exe——也就是你运行 C# 程序时，Windows 真正启动的那个原生 EXE。”不太懂，运行c#程序时发生了什么？是不是在运行前我得先装 dotnet.ext 环境？这个环境是什么如何理解。

元宝:

你这个问题问得非常关键——上一轮我说"dotnet.exe 是房东"其实说得有点过于绝对，让我重新给你捋一遍，把你困惑的几个点一次性讲透。

一、先回答最核心的问题：运行 C# 程序前，要不要装"dotnet 环境"？

要的。 但更准确的说法是：要在机器上运行 C# 程序，这台机器必须有**对应的 .NET 运行时（Runtime）**。

为什么？因为 C# 代码编译后不是 CPU 能直接执行的机器码，而是一种叫 **CIL（通用中间语言）** 的东西。CPU 不认识 CIL，必须有个"翻译官"在程序运行的时候把 CIL 一句句翻译成机器码——这个翻译官就是 **CLR（公共语言运行时）**，而 CLR 就包含在 .NET 运行时里。

> 📌 所以"装 dotnet 环境"的本质 = **在机器上安装 CLR + 基础类库**，让这台机器具备"执行 CIL"的能力

微软官方的下载页把这件事说得很清楚：

- **.NET Runtime**：只包含运行一个控制台程序所需的最小组件
- **.NET Desktop Runtime**：运行 Windows 桌面程序（WinForms/WPF）所需
- **ASP.NET Core Runtime**：运行 Web/服务端程序所需
- **.NET SDK**：开发用的，= Runtime + 编译器 + 命令行工具（`dotnet build` / `dotnet run` 等）

开发时你装 SDK 就行，SDK 里已经包含了 Runtime。部署到生产服务器时，如果是"框架依赖部署"，服务器上就只要装对应的 Runtime。

二、"运行环境"到底是什么？里面有什么？

可以把 .NET 运行环境想象成一个**工具箱**，里面至少有这三样东西：

```
.NET 运行环境
├── CLR（公共语言运行时）
│   ├── JIT 编译器    —— 把 CIL 翻译成本地机器码
│   ├── GC            —— 自动管理内存
│   ├── 线程池        —— 调度托管线程
│   └── 类型系统      —— 类型安全校验
│
```

```
    ├── BCL（基础类库）
    │    └── System.IO、System.Collections、System.String……
    │        你代码里 using 的那些 System.* 都在这里
    │
    └── 宿主入口（dotnet.exe / hostfxr.dll 等）
        └── 负责把 CLR "叫醒"并交给它你的程序
```

> 💡 注意：在 .NET Framework 时代，运行环境是**跟着 Windows 走**的——大多数 Windows 系统已经预装了 .NET Framework，所以你双击一个 .NET Framework 的 exe 往往能直接跑，不用另装。而现代的 .NET 5+ 是**独立发布**的，需要单独安装（或者用"自包含部署"把运行时打包进你的程序文件夹）。

三、运行一个 C# 程序时，到底发生了什么？

这是你最困惑的地方。我分 **.NET Framework** 和 **现代 .NET** 两种情况讲，因为它们略有不同。

情况 A：运行一个 .NET Framework 的 exe（传统 Windows 程序）

```
① 你双击 MyApp.exe
        ↓
② Windows 加载器检查 EXE 文件头，发现 COM Descriptor Directory 位被置位
   → 这说明"我是托管模块，不是普通原生程序"
        ↓
③ Windows 加载 MsCorEE.dll（这就是上一轮说的"垫片"）
        ↓
④ MsCorEE.dll 把 EXE 的入口点改成 CLR 的入口，
   然后调用 _CorExeMain
        ↓
⑤ CLR 被初始化：
   · 加载 clr.dll
   · 创建系统域、共享域、默认 AppDomain
   · 找到 MyApp.exe 里的 Main 方法
        ↓
⑥ JIT 编译器把 Main 方法的 CIL 翻译成本地机器码
        ↓
⑦ 你的代码终于开始执行！
```

微软文档原话："操作系统加载器通过检查 COFF 头里的 COM Descriptor Directory 位来识别托管模块。如果识别到托管模块，它会加载 MsCorEE.dll……将映像的入口点改为 CLR 的入口点"。

关键点：这种情况下，**没有 dotnet.exe 什么事**。Windows 直接通过 MsCorEE.dll 把 CLR 拉起来，你的 exe 本身就是"宿主"。

情况 B：运行一个现代 .NET 的 dll（`dotnet MyApp.dll`）

现代 .NET 跨平台，不能指望每个 OS 都能像 Windows 那样识别托管 exe，所以微软设计了一条 **CoreHost 链式调用**：

```
① 你在命令行敲：dotnet MyApp.dll
        ↓
② dotnet.exe（位于 C:\Program Files\dotnet\dotnet.exe）
   作为 CoreHost 链的入口被启动
        ↓
③ dotnet.exe 调用 hostfxr.dll
   （Host FX Resolver，负责解析用哪个版本的运行时）
        ↓
④ hostfxr.dll 根据 MyApp.dll 的 .deps.json 等配置，
   选定合适的 .NET 版本，调用 hostpolicy.dll
        ↓
⑤ hostpolicy.dll 加载所有依赖：
   · 你的 MyApp.dll
   · Newtonsoft.Json.dll 等第三方库
   · System.Console.dll 等运行时基础库
        ↓
⑥ 最终加载 coreclr.dll —— CLR 本体正式启动
        ↓
⑦ JIT 把 Main 的 CIL 翻译成本地机器码
        ↓
⑧ 你的代码开始执行
```

可以看到，`dotnet.exe` **在这里扮演的就是"宿主"角色**——它负责把 CLR 请进进程。上一轮我说"dotnet.exe 是最常见的房东"，指的就是这种现代 .NET 的场景。

> 📌 一句话区分：
>
> - **.NET Framework**：你的 exe 自己就是宿主，通过 MsCorEE.dll 拉起 CLR
> - **.NET 5+**：`dotnet.exe` 是宿主，它拉起 CLR 后再加载你的 dll

四、回到"CLR 寄宿"这个概念

现在你回头看《CLR via C#》第22.1节说的"CLR 寄宿"，就不抽象了：

"寄宿"= 把一个进程作为宿主，把 CLR 这个 DLL 加载进自己的内存空间，然后由 CLR 来执行托管代码。

常见的宿主有：

- **dotnet.exe** —— 命令行运行 .NET 程序时的宿主
- **IIS 的 w3wp.exe** —— ASP.NET 程序的宿主
- **SQL Server 的 sqlservr.exe** —— SQL CLR 的宿主
- **你自己写的 C++ 程序** —— 理论上任何原生程序都能通过 COM 接口把 CLR 请进来，这正是 22.1 节讲的那套 `CLRCreateInstance → GetRuntime → GetInterface(ICLRRuntimeHost)` 流程

五、所以"我写 C# 程序，要准备什么？"

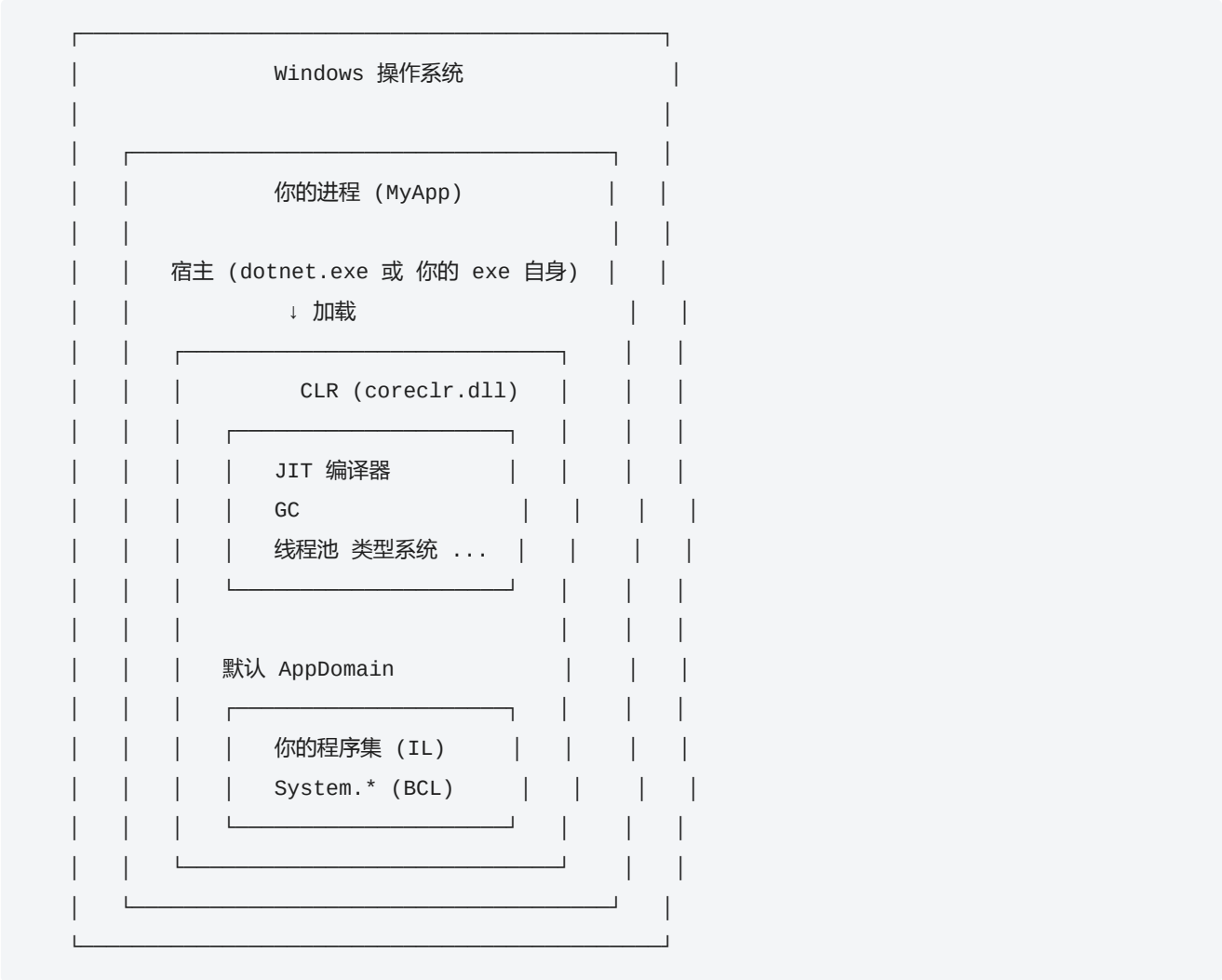
如果你是**开发**：

1. 装 **.NET SDK**（包含 Runtime + 编译器 + `dotnet` 命令行）
2. 写代码 → `dotnet build` 编译成 dll/exe
3. `dotnet run` 运行，或者把编译产物拷到别的机器上跑

如果你是**部署到一台干净机器上运行**：

- 方式一：**框架依赖部署** —— 目标机先装对应版本的 .NET Runtime，然后拷贝你的程序文件过去运行
- 方式二：**自包含部署** —— 发布时勾选"自包含"，Runtime 会被打包进你的程序文件夹，目标机**零依赖**，双击就能跑（代价是文件体积大）

一张图收尾



从左往右看：Windows 创建进程 → 宿主把 CLR 请进进程 → CLR 创建 AppDomain 并加载你的程序集 → JIT 把 IL 翻成机器码 → CPU 真正执行。

> 🎯 记住这 3 句话，这一节就算过关了：
>
> 1. **C# 编译出来的是 CIL，不是机器码**，所以机器上必须有 CLR 才能运行
> 2. **"装 dotnet 环境" = 装 CLR + 基础类库**，开发装 SDK，运行装 Runtime
> 3. **宿主负责把 CLR 请进进程**：.NET Framework 下宿主是你的 exe 自身；.NET 5+ 下宿主是 dotnet.exe

下一节（22.2 AppDomain）就会讲：CLR 被请进进程后，是怎么在你的程序外面再套一层"逻辑容器"来做隔离的——这正是上图里那个"默认 AppDomain"框框的由来。要继续听我用这种方式讲 22.2 吗？

▾ 引用 18 篇资料作为参考

1. 🔗 How .NET Really Works: From Build to CLR Execution Explained Simply
2. 🅒 Microsoft.NET Runtime是什么
3. 🔗 What is .NET Framework? A software development framework .NET
4. 🔗 .Net技术栈梳理01-核心框架与运行时CLR
5. 🔗 .NET
6. .NET Framework vs .NET Core:一场从“温室”到“旷野”的进化
7. 🔗 C# 入口点机制深度解构:从 CLR 调度到顶级语句的演变
8. ASP.NET Core 时代只需要“有运行时”,运行时是什么
9. 🔗 解构 .NET:一份关于微软软件生态演进的考古报告 TrueSight
10. 🔗 Managed Execution Process - .NET
11. 🔗 .NET-Architekturkomponenten.NET architectural components
12. 🔗 .NET与.NET Framework有何本质区别?
13. C#/.NET程序运行流程
14. 🔗 .NET Platform
15. 🅒 新建控制台项目时,选‘控制台应用’和‘控制台应用(.NET Framework)’到底差在哪?
16. 🔗 _CorValidateImage Function - .NET Framework
17. 🔗 Download .NET 10.0 (Linux, macOS, and Windows) .NET
18. 🔗 .NET implementations - .NET

用户:

装C#环境，是不是就是装 .net runtime，其包含了 CLR，一般是在系统默认盘里增加了一些exe文件；装了 .net runtime 后，运行自己写的 exe 时，自动调用系统默认盘的 .net runtime 环境加载并创建clr实例，这里创建相当于复制一份 clr 到进程里运行自己写的 exe；这样理解对吗？

元宝:

你理解的方向**基本对了**，但有两个细节需要修正，修正完之后你的模型就完全准确了：

✅ 你对的部分

1. **"装 C# 环境 ≈ 装 .NET Runtime"** —— 对的。Runtime 里包含了 CLR（核心执行引擎）+ 基础类库（BCL）
2. **".NET Runtime 文件一般在系统盘"** —— 对的。比如 .NET Framework 装在 `C:\Windows\Microsoft.NET\Framework64\v4.0.30319\` 下，里面有 `clr.dll`、`mscoreei.dll` 等；现代 .NET 装在 `C:\Program Files\dotnet\` 下
3. **"运行自己写的 exe 时，自动调用系统的 .NET Runtime 来加载并创建 CLR 实例"** —— 大方向对的

❌ 需要修正的两个点

修正一：不是"复制一份 CLR 到进程里"，而是"加载（Load）"

你用的词是"复制一份 clr 到进程里"——这个说法容易误导。准确的机制是：

> **CLR 的 DLL 文件本身还在系统盘的原位置**，运行时通过 **内存映射（Memory Mapping）** 把这个 DLL 的代码和数据**映射**到你的进程地址空间里。

打个比方：

- ❌ "复制" = 把 `C:\Windows\...\clr.dll` 拷贝一份到你 exe 的文件夹里
- ✅ "加载/映射" = 系统盘的 `clr.dll` 还是那份文件，但你的进程通过**虚拟地址**直接"看到"并"使用"它里面的代码

这就是为什么几万个 .NET 程序同时运行时，系统盘里**只有一份** `clr.dll` 文件，但每个进程都有自己独立的 CLR 实例——它们**共享同一份代码（只读），但每个进程有自己独立的数据（堆、栈、GC 状态等）。**

所以更准确的表达是：

> **"装了 .NET Runtime 后，运行自己写的 exe 时，系统会自动把 Runtime 里的 `clr.dll` 加载（映射）到你的进程地址空间中，并在进程内初始化出一个 CLR 实例来执行你的代码。"**

修正二：不是所有 exe 都"自动"调用 Runtime

这取决于你的 exe 是哪种类型：

情况 A：.NET Framework 的 exe

- 你的 exe 文件头里带有一个特殊的"托管"标记
- Windows 加载这个 exe 时**一眼就能看出来**"哦，这是托管的"，于是自动加载 `MsCorEE.dll` 这个垫片，垫片再去加载 `clr.dll`
- 看起来就像"双击 exe 自动调用 Runtime"——你感知不到中间过程

情况 B：现代 .NET（.NET 5/6/7/8...）的 exe

- 如果你的 exe 是 **framework-dependent** 发布的，双击它可能**不直接跑**——它依赖于 `dotnet.exe` 这个宿主
- 实际运行方式往往是：`dotnet.exe` 先启动 → 它去加载 `clr.dll` → 然后通过 CLR 加载并执行你的 exe
- 当然，你也可以发布成 **self-contained** 模式，把 Runtime 一起打包，这样双击 exe 就能直接跑，不依赖系统装的 Runtime

---

🎯 修正后的完整正确理解

把你原来的说法修正一下，就是完全准确的版本：

> **装 C# 环境 = 在系统盘安装 .NET Runtime，其中包含 CLR（clr.dll）和基础类库。当我运行自己写的 exe 时：**
>
> - **如果是 .NET Framework 程序**：Windows 识别到 exe 是托管的，自动通过 `MsCorEE.dll` 垫片把系统盘里的 `clr.dll` **加载（内存映射）到我的进程地址空间**，并在进程内初始化出一个 CLR 实例来 JIT 编译和执行我的 C# 代码
> - **如果是现代 .NET 程序**：通常由 `dotnet.exe` 作为宿主，它把 `clr.dll` 加载进来，再通过 CLR 执行我的程序
>
> **CLR 的代码在系统盘只有一份（共享），但每个进程都有自己独立的 CLR 实例（拥有独立的堆、GC、线程池等状态）**

💡 一个帮你巩固的小实验

你可以在 Windows 上做这个实验：

1. 装好 .NET Framework（Win10/11 大部分已预装）
2. 写一个最简单的 C# 程序编译成 exe
3. **双击运行**
4. 打开任务管理器 → 详细信息 → 右键列标题 → 勾选"命令行"
5. 你会发现：你的 exe 进程**本身就直接运行着**，但在它的内存空间里已经加载了 `clr.dll`

或者更直观的：

- 打开 Process Explorer（Sysinternals 工具）
- 找到你的 exe 进程
- 查看它加载的 DLL 列表
- 你能看到 `clr.dll`、`mscoreei.dll`、`System*.dll` 等都在里面

这就证明了：**CLR 是被"加载"进你的进程，而不是"复制"了一份文件过去。**

你的直觉其实已经摸到门道了，只是"复制"这个词换成"加载/映射"就更精确。这种"系统有一份代码，多个进程共享这份代码但各有各的数据"的模式，在 Windows 里非常常见——比如 `kernel32.dll`、`user32.dll` 这些系统 DLL 也是同样的机制。CLR 本质上就是一个**比较特殊的系统级 DLL**。

用户:

你总结的”22.2 AppDomain“也解释一下，我不太懂。

元宝:

好，接着用"零基础视角"把 22.2 AppDomain 讲透。上回我们说到 CLR 已经被"请"进了进程，接下来 CLR 就要在自己的"地盘"里再划出一块块**逻辑隔离区**——这就是 AppDomain。

一、先建立直觉：为什么需要 AppDomain？

想象一个场景：你写了一个 ASP.NET 网站，IIS 上同时部署了 **10 个站点**，每个站点都是不同的 C# 程序。

如果没有 AppDomain，会怎样？

- 这 10 个站点的代码**全在一个进程里跑**
- 站点 A 里的一个 bug 把内存写崩了 → **站点 B/C/D…全挂**
- 你想更新站点 A 的代码 → **必须停掉整个进程**，其他 9 个站点也得跟着下线
- 站点 A 加载了一个恶意插件 → **可以随意访问站点 B 的数据**

这显然不行。那怎么办？最粗暴的办法是：**每个站点开一个独立进程**。但进程是很贵的——创建慢、占用内存大、进程间通信复杂。

> 💡 **AppDomain 就是微软给出的答案**：在**同一个进程里**模拟出多个"逻辑进程"，它们之间彼此隔离，但开销远小于真实进程。

可以这么类比：

真实进程	AppDomain
一栋独立的房子	同一栋楼里的不同公寓
有自己的地址空间	共享进程地址空间，但逻辑隔离
创建开销大	创建开销小
一个崩了不影响其他	一个崩了……理论上不影响其他
卸载 = 杀掉进程	卸载 = `AppDomain.Unload()`

二、AppDomain 到底是什么？

定义：AppDomain 是 CLR 提供的一种**逻辑容器**，用来装一组程序集（Assembly）。它是**隔离的单元、卸载的单元、权限的单元、配置的单元。**

CLR 启动时，会**自动创建一个 AppDomain**，叫**默认 AppDomain（Default AppDomain）**。你的 `Main` 方法就跑在这个默认 AppDomain 里。

但你完全可以自己再创建更多的 AppDomain：

```
    // 在当前进程里再建一个 AppDomain
    AppDomain newDomain = AppDomain.CreateDomain("MyPluginDomain");

    // 在新 AppDomain 里加载一个程序集并执行
    newDomain.ExecuteAssembly("Plugin.dll");
```

```
// 用完就扔——整个 AppDomain 被卸载，里面的所有代码、对象、内存全部回收
AppDomain.Unload(newDomain);
```

> 📌 关键点：**AppDomain 可以卸载，但进程不能。**一个进程里可以有多个 AppDomain，卸载其中一个，其他的不受影响，进程本身也不受影响。

三、AppDomain 提供的四大能力

书里总结了 AppDomain 的四个核心特性，我一个个讲：

1. 代码隔离（Isolation）

不同 AppDomain 里的代码**不能直接互相访问对象。**

```
// 在默认 AppDomain 里
var obj = new MyClass();  // MyClass 在默认 AppDomain

// 在另一个 AppDomain 里
// 无法直接拿到上面那个 obj！
```

为什么？因为 AppDomain 是"逻辑进程"，跨 AppDomain 访问对象就像跨进程——必须经过**封送（Marshaling）**。这就引出了下面的重点。

2. 可卸载（Unloadable）

`AppDomain.Unload(domain)` 可以把整个 domain 卸掉，里面所有的：

- 程序集
- 类型对象
- 已创建的对象实例
- 分配的堆内存

……**全部被 GC 回收。**这是 AppDomain **最有价值的能力**——你可以动态加载插件，用完即弃，不污染主程序。

3. 可保护（Configurable Security）

创建 AppDomain 时可以给它指定一个**权限集（Permission Set）**。比如你可以创建一个"受限 AppDomain"，里面的代码：

- ❌ 不能访问文件系统
- ❌ 不能访问网络
- ❌ 不能调用 P/Invoke

这就实现了**沙箱（Sandbox）**——用来跑不可信的第三方代码。

4. 可配置（Configurable）

每个 AppDomain 可以有不同的配置：

- **搜索路径**：去哪里找程序集
- **版本绑定重定向**：程序集版本冲突时怎么处理
- **卷影复制（Shadow Copy）**：允许更新正在被使用的 DLL
- **加载器优化**：性能调优

这就是为什么 IIS 上每个站点可以有自己独立的 `web.config` 和 `bin` 目录——它们各自跑在各自的 AppDomain 里。

四、跨 AppDomain 访问：封送（Marshaling）

这是 22.2 节最难懂的部分。既然 AppDomain 之间不能直接访问对象，那如果我想从主程序调用插件里的一个对象的方法，怎么办？

CLR 提供了**两种封送方式**：

方式一：按引用封送（Marshal-by-Reference）

对象必须**继承** `MarshalByRefObject`：

```
// 在插件程序集里
public class Plugin : MarshalByRefObject
{
    public string DoWork() => "插件干活中...";
}

// 在主程序里
AppDomain pluginDomain = AppDomain.CreateDomain("Plugin");
Plugin proxy = (Plugin)pluginDomain.CreateInstanceAndUnwrap(
    "PluginAssembly",
    "Plugin");

string result = proxy.DoWork();  // 跨 AppDomain 调用
```

`proxy` 不是真正的 `Plugin` 对象，而是一个**代理（Proxy）**。当你调用 `proxy.DoWork()` 时：

1. CLR 拦截这次调用
2. 把调用参数**封送**到 `pluginDomain`
3. 在 `pluginDomain` 里执行真正的 `DoWork()`
4. 把返回值**封送**回主 AppDomain
5. 你拿到结果

> ⚠️ 代价：**每次跨边界调用都有性能开销**（序列化/反序列化 + 上下文切换）

方式二：按值封送（Marshal-by-Value）

对象**不继承** `MarshalByRefObject`**，但标记为** `[Serializable]`：

```
[Serializable]
public class Data
{
    public int Value { get; set; }
}

// 主程序里
AppDomain pluginDomain = AppDomain.CreateDomain("Plugin");
Data data = new Data { Value = 42 };

// 跨 AppDomain 传递 data 时，CLR 会：
// 1. 把 data 序列化成字节流
// 2. 在目标 AppDomain 里反序列化
// 3. 创建 data 的一个"精准复制"对象
// 4. 把这个副本交给你
```

这种方式下，目标 AppDomain 拿到的是**数据的副本**，不是引用。之后两边各改各的，互不影响。

怎么选？

场景	选择
对象有行为（方法），想远程调用	按引用（`MarshalByRefObject`）
对象只是数据，想传过去自己用	按值（`[Serializable]`）
对象很大，不想复制	按引用
对象很小，且不需要跨域共享状态	按值

五、Loader 堆：AppDomain 的"记忆"

这是 Richter 讲得最深的部分，但理解了会很通透。

每个 AppDomain 都有自己的 Loader 堆。Loader 堆里存的是：这个 AppDomain **自创建以来访问过的所有类型**的类型对象（Type Object）。

什么意思？看这个例子：

```
进程里有两个 AppDomain：DomainA 和 DomainB

DomainA 里第一次用到 string → CLR 在 DomainA 的 Loader 堆里
                                    创建 System.String 的类型对象
```

```
DomainB 里第一次用到 string → CLR 在 DomainB 的 Loader 堆里
                                也创建 System.String 的类型对象
```

两个 AppDomain 里的 `System.String` 类型对象，在内存里是两份！ JIT 编译出的本地代码也是两套。

为什么这么"浪费"？为了**隔离性**——这样卸载 DomainA 时，可以直接把 DomainA 的 Loader 堆整个清掉，不用担心影响到 DomainB。

> 💡 **唯一的例外**：`MSCorLib.dll`（含 `System.Object`、`System.Int32`、`System.String` 等最基础的类型）以 **AppDomain 中立（AppDomain-neutral）**方式加载——所有 AppDomain **共享同一份**类型对象和 JIT 代码。CLR 为它维护一个特殊的 Loader 堆。

这是为了性能优化：这些基础类型被所有 AppDomain 频繁使用，如果每域一份就太浪费了。

六、卸载 AppDomain：背后发生了什么？

调用 `AppDomain.Unload(domain)` 时，CLR 做了一连串动作（这部分书里讲得很细）：

```
① 挂起进程中所有曾执行过托管代码的线程
② 检查所有线程栈，找出哪些线程：
   - 正在该 AppDomain 中执行，或
   - 将来可能返回到该 AppDomain
③ 对这些线程强制抛出 ThreadAbortException
   → 线程展开调用栈，执行 finally 块做清理
   → 但如果在执行 finally/catch/类构造函数/非托管代码，
     则等这些代码块执行完再抛
④ 所有相关线程离开该 AppDomain 后：
   → 给所有指向该 AppDomain 对象的代理打上"失效"标记
   → 之后再调用这些代理的方法会抛 AppDomainUnloadedException
⑤ 强制执行一次 GC，调用待回收对象的 Finalize 方法
⑥ 恢复其余线程
```

> ⚠️ 注意：**`AppDomain.Unload` 是个阻塞调用**，它会等所有清理工作做完才返回。

七、监视 AppDomain 的资源使用

CLR 提供了**应用程序域资源监视（ARM）**机制，可以监控每个 AppDomain 消耗了多少 CPU 时间和内存：

```
// 打开全局监视开关
AppDomain.MonitoringIsEnabled = true;

// 读取当前 AppDomain 的资源使用
TimeSpan cpu = AppDomain.CurrentDomain.MonitoringTotalProcessorTime;
long memoryUsed = AppDomain.CurrentDomain.MonitoringSurvivedMemorySize;
long memoryAllocated = AppDomain.CurrentDomain.MonitoringTotalAllocatedMemorySize;
```

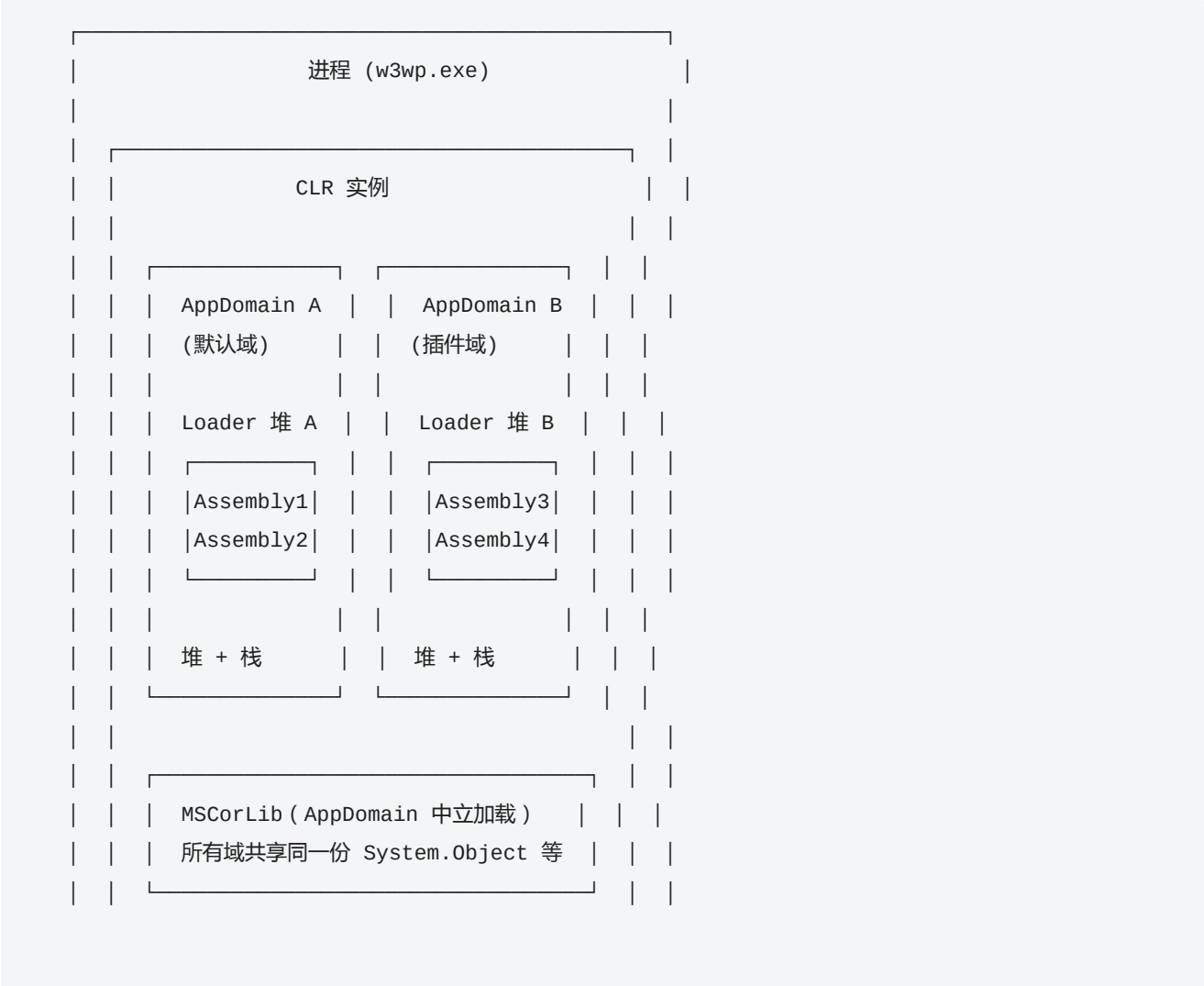
Richter 在书里写了一个 `AppDomainMonitorDelta` 工具类（实现 `IDisposable`），可以方便地测量**某段代码执行前后** AppDomain 的资源增量。这是诊断内存泄漏、性能瓶颈的好工具。

八、重要提醒：.NET Core / .NET 5+ 里的 AppDomain

> ⚠️ **AppDomain 是 .NET Framework 时代的产物**。在 .NET Core 和 .NET 5/6/7/8 里：
>
> - `AppDomain` 类还在，但**大部分隔离/卸载功能已被阉割**
> - **不支持**创建新的 AppDomain
> - **不支持** `AppDomain.Unload()`
> - 取而代之的是 **`AssemblyLoadContext`**——它只提供"程序集卸载"的能力，不提供完整的 AppDomain 隔离

所以如果你在用 .NET 5+ 开发，这一章的知识更多是**理解历史和原理**。但如果你维护 .NET Framework 项目（很多企业还在用），AppDomain 依然是核心机制。

九、一张图收尾



```
|  └──────────────────────────────────────┘  |
└────────────────────────────────────────────┘
```

🎯 记住这 5 句话，22.2 节就过关了

1. **AppDomain 是 CLR 在进程内划分的"逻辑容器"**，用来隔离程序集
2. **一个进程可以有多个 AppDomain**，CLR 启动时自动创建"默认 AppDomain"
3. AppDomain 提供四大能力：**代码隔离、可卸载、可保护、可配置**
4. 跨 AppDomain 访问对象有两种方式：**按引用封送**（`MarshalByRefObject`）和**按值封送**（`[Serializable]`）
5. **每个 AppDomain 有自己的 Loader 堆**，`MSCorLib` 除外（AppDomain 中立加载）；**.NET Core/.NET 5+ 已用 `AssemblyLoadContext` 替代 AppDomain**

💡 一个现实中的 AppDomain 应用场景

ASP.NET 的经典模型（.NET Framework 时代）：

- IIS 的 `w3wp.exe` 进程里，**[每个站点运行在独立的 AppDomain 里]**
- 站点 A 更新 dll → 只需卸载站点 A 的 AppDomain，其他站点不受影响
- 站点 A 的代码崩了 → 异常被限制在站点 A 的 AppDomain 内，不会搞垮整个 IIS
- 站点 A 是可疑的第三方代码 → 用受限权限集创建 AppDomain，沙箱化运行

这就是 AppDomain 设计的初衷——**让一个进程能安全地跑多个互相不信任的工作负载。**

学到这里，你对 .NET 的"进程 → CLR → AppDomain → 程序集"四层模型应该就有完整画面了。下一章（第23章）讲"程序集加载和反射"，正好接着 Loader 堆和 `AssemblyLoadContext` 的话题往下挖。需要继续吗？

用户:

你说的站点a、b... 是什么？

元宝:

好问题——我用"站点 A、站点 B"其实偷懒了，没解释清楚。这里说的"站点"指的是 **IIS 上部署的 ASP.NET Web 应用程序**（网站）。

具体来说

IIS（Internet Information Services）是 Windows 上的 Web 服务器。你可以在一台服务器的 IIS 上同时部署多个网站：

- **站点 A** → `www.myblog.com`（一个博客网站）

- **站点 B** → `www.shop.com`（一个电商网站）
- **站点 C** → `api.myservice.com`（一个 Web API 服务）

每个站点本质就是一个 **ASP.NET 应用程序**，背后是一堆 C# 代码编译出来的程序集。

它们和 AppDomain 的关系

在 **.NET Framework 时代**的经典 IIS 模型里：

> **IIS 会启动一个工作进程 `w3wp.exe`，然后为部署在该服务器上的每个 ASP.NET 站点创建一个独立的 AppDomain，并在这个 AppDomain 里加载对应站点的程序集。**

```
w3wp.exe 进程
└── CLR
    ├── AppDomain: 站点A (www.myblog.com)
    │   └── 加载 Blog.dll, EntityFramework.dll ...
    ├── AppDomain: 站点B (www.shop.com)
    │   └── 加载 Shop.dll, EntityFramework.dll ...
    └── AppDomain: 站点C (api.myservice.com)
        └── 加载 Api.dll ...
```

这样设计的好处（也就是上一轮我说的那几点）：

1. **站点 A 崩溃了** → 只有 AppDomain A 里的对象受影响，站点 B/C 继续正常服务
2. **更新站点 A 的代码** → 只需卸载 AppDomain A 再重建，B/C 不停机
3. **站点 A 是第三方不可信代码** → 用受限权限集创建 AppDomain A，沙箱化

如果你没接触过 IIS/ASP.NET

没关系，换个更生活化的比喻：

> 把 `w3wp.exe` 进程想象成一栋**写字楼**
> 把 **AppDomain** 想象成写字楼里的**一间间办公室**
> 把**站点 A/B/C** 想象成租在这栋楼里的**三家公司**

三家公司共用这栋楼的水电（进程的地址空间、CLR 的服务），但每家有自己的办公室（AppDomain），互不干扰。一家公司装修（更新代码）不影响其他两家，一家着火（程序崩溃）也不会烧到邻居。

如果你写的是普通的 C# 控制台程序

那你的情况更简单：

```
MyApp.exe 进程
└── CLR
```

```
        └── 默认 AppDomain（唯一一个）
            └── 加载 MyApp.exe, Newtonsoft.Json.dll ...
```

只有一个 AppDomain，就是 CLR 启动时自动创建的"默认 AppDomain"。你写的 `Main` 方法就跑在这里面。

除非你主动调用 `AppDomain.CreateDomain(...)` 去创建新的 AppDomain（比如你要做一个插件系统，动态加载/卸载插件），否则你的程序里**永远只有一个 AppDomain**。

所以"站点 A、站点 B"在那个例子里就是：

部署在同一个 IIS 服务器上的不同 ASP.NET Web 应用——仅此而已。我用它们做例子是因为 AppDomain 最经典的应用场景就是 ASP.NET：IIS 用 AppDomain 来隔离多个站点，让它们在同一进程里安全共存。

如果你做的是桌面程序、控制台程序、WPF 程序，通常**用不到多个 AppDomain**，CLR 默认的这一个就够你用了。但理解 AppDomain 的概念，对你理解 CLR 的整体架构非常有帮助——它解释了"为什么 ASP.NET 能做到不停机更新"、"为什么 SQL Server 能安全地跑 CLR 存储过程"等很多 .NET 生态里的现象。

用户:

appdomain不太了解，其和程序集有什么关系？是一个程序集对应一个appdomain吗？

元宝:

不是的，**AppDomain 和程序集不是一一对应的关系。**准确的关系是：

> **AppDomain 是容器，程序集是被装进容器的东西。一个 AppDomain 可以装多个程序集；同一个程序集也可以被多个 AppDomain 加载。**

微软官方文档说得很明确："在可以执行程序集中所包含的代码之前，必须将程序集加载到应用程序域中。运行普通的应用程序会导致**几个程序集加载到一个应用程序域中**"。

用图来理解三者关系

```
进程 (w3wp.exe / MyApp.exe)
│
└── CLR
    │
    └── AppDomain A（默认 AppDomain）
    │   ├── MyApp.exe          ← 你的主程序集
    │   ├── System.dll         ← 基础类库
    │   ├── System.Core.dll
    │   ├── Newtonsoft.Json.dll ← 第三方库
    │   └── ...                ← 还有其他被引用的程序集
    │
    └── AppDomain B（你手动创建的）
        ├── PluginX.dll        ← 插件程序集
```

```
            └── System.dll          ← 注意：这个 System.dll
                                      在 B 域里是另一份
```

可以看到：

- **AppDomain A 里装了 4+ 个程序集** —— 一对多
- **System.dll 同时被 AppDomain A 和 B 加载** —— 一个程序集被多个域加载

关键点：程序集必须"装进" AppDomain 才能运行

程序集（.dll / .exe）本身是静态的文件，**它自己不能执行。**必须有人把它"加载到"一个 AppDomain 里，CLR 才会为它创建类型对象、JIT 编译它的方法，代码才能真正跑起来。

所以正确的理解顺序是：

1. **进程启动** → Windows 创建进程
2. **CLR 被加载** → 进程里有了 CLR 实例
3. **CLR 创建默认 AppDomain** → 有了一个"容器"
4. **程序集被加载进 AppDomain** → 你的 `MyApp.exe` 以及它依赖的所有 dll 都被装进这个默认 AppDomain
5. **CLR 执行程序集里的代码** → 你的 `Main` 方法开始跑

一个程序集被多个 AppDomain 加载时会发生什么？

这是 Richter 书里讲 Loader 堆时提到的重点。同一个程序集（比如 `System.dll`）被两个 AppDomain 加载：

默认情况（非域中立加载）：

- 每个 AppDomain 里**各有一份**该程序的类型对象和 JIT 编译代码
- 好处：卸载 AppDomain A 时，A 里的 `System.dll` 可以跟着一起被回收
- 代价：内存里有多份副本，费内存

域中立加载（Domain-Neutral）：

- 所有 AppDomain **共享同一份** JIT 编译代码
- 好处：省内存
- 代价：**这个程序集永远不能从进程里卸载**，除非整个进程退出

`MSCorLib.dll`（含 `System.Object`、`System.Int32` 等最基础的类型）**始终以域中立方式加载**——所有 AppDomain 共享它，且永远不能卸载。

回到你的疑问："一个程序集对应一个 AppDomain 吗？"

不是。 正确的对应关系有三种可能：

场景	关系
普通控制台程序	1 个 AppDomain（默认域）⟷ **多个程序集**（你的 exe + 所有依赖的 dll）
插件系统	1 个程序集 ⟷ **多个 AppDomain**（主域加载一份，插件域也加载一份）
简单 ASP.NET 站点	1 个 AppDomain ⟷ **多个程序集**（站点 dll + Framework 类库）

最常见的情况是第一种：你写一个简单的 C# 控制台程序，运行起来后：

```
MyApp.exe 进程
└── CLR
    └── 默认 AppDomain（唯一一个）
        ├── MyApp.exe
        ├── System.Runtime.dll
        ├── System.Console.dll
        ├── System.Collections.dll
        └── ...（十几个甚至几十个程序集）
```

一个 AppDomain，装了十几个程序集。这就是常态。

一个帮助理解的日常比喻

把 AppDomain 想象成**书架**，把程序集想象成**书**：

- 一个书架上可以放**很多本书**（一个 AppDomain 加载多个程序集）✅
- 同一本书可以被**多个书架同时存放**（一个程序集被多个 AppDomain 加载）✅
- 书自己不会"读自己"，必须放到书架上，有人（CLR）去翻它，知识（代码）才能被使用 ✅
- 你想下架某本书，直接把整个书架搬走（卸载 AppDomain）就行 ✅

⚠️ .NET Core / .NET 5+ 的情况

如果你用的是现代 .NET，要注意：

> 在 .NET Core 上，AppDomain 的实现受设计限制，**不提供隔离、卸载或安全边界**。.NET Core 中**只有一个 AppDomain**。隔离和卸载通过 `AssemblyLoadContext` 实现。

所以：

- 学《CLR via C#》（.NET Framework 语境）→ AppDomain 是核心概念，一个进程多个 AppDomain，每个装多个程序集
- 用 .NET 5+ 开发 → AppDomain 基本退化成"只有一个"，替代方案是 `AssemblyLoadContext`（它更接近"一个程序集对应一个上下文"的粒度）

> 🎯 **记住这句话**：AppDomain 是**装程序集的容器**，不是和程序集一一对应的。一个普通 C# 程序运行时，**一个默认 AppDomain 里装着几十个程序集**——你的 exe 只是其中之一。

▾ 引用 10 篇资料作为参考

1. 🔗 应用程序域和程序集 Microsoft Learn
2. 🔗 Application Domains and Assemblies
3. 🔗 AppDomain与进程、线程、Assembly之间关系
4. 🔗 《CLR via C#》笔记——AppDomain(1)
5. 🔗 AppDomain 类 (System)
6. 🔗 CLR via C# - Chapter 23 Assembly Loading and Reflection
7. 🔗 应用程序域
8. 🔗 .NET AppDomain、Process 、Context
9. 🔗 C#里的应用程序域AppDomain
10. AppDomain

用户:

你还提到 net 5+后appdomain被逐步阉割弃用？这方面展开说说。

元宝:

好问题，这一节我前面讲得有点笼统，说是"阉割弃用"，但其实准确情况更微妙一些。我把 .NET Framework 和 .NET 5+ 的差异完整展开讲。

一、先纠正一个常见误解：**AppDomain 不是被"完全删除"**

很多文章说".NET 5+ 完全移除了 AppDomain"——这个说法**不准确**。

真相是：`AppDomain` **这个类还在**，你依然可以写 `AppDomain.CurrentDomain` 拿到当前域，访问它的 `BaseDirectory`、`GetAssemblies()` 等方法。但是：

> ⚠️ 在 .NET Core 和 .NET 5+ 上，AppDomain 的实现**受设计限制，不提供隔离、卸载或安全边界。这些版本只有一个 AppDomain**。

微软官方文档原话："On .NET Core and .NET 5+, the AppDomain implementation is limited by design and does not provide isolation, unloading, or security boundaries. **These versions have exactly one AppDomain**. Isolation and unloading are provided through AssemblyLoadContext."

换句话说：

能力	.NET Framework	.NET 5+
`AppDomain.CurrentDomain`	✅ 可用	✅ 可用（只有这一个）
`AppDomain.CreateDomain()`	✅ 创建新域	❌ 抛出 `PlatformNotSupportedException`
`AppDomain.Unload()`	✅ 卸载指定域	❌ 不支持
隔离/安全边界	✅ AppDomain 提供	❌ 不提供
动态加载程序集	AppDomain 内加载	改用 `AssemblyLoadContext`

所以准确说法是：**AppDomain 的"隔离容器"语义在 .NET 5+ 中被拿掉了，只剩下一个"当前域"的壳。**

微软的 .NET 6+ 移植文档说得更直白："不支持创建更多应用域，并且**将来没有添加此功能的计划。**对于代码隔离，请使用单独的进程或容器作为替代方法。若要动态加载程序集，请使用 AssemblyLoadContext 类。"

二、为什么微软要"砍掉"AppDomain？

这不是拍脑袋决定的，有三个深层原因：

1. AppDomain 依赖"远程处理（Remoting）"，而 Remoting 被砍了

AppDomain 之间跨域访问对象，底层靠的是 **Remoting** 技术（也就是 `MarshalByRefObject` 那一套）。.NET Core 重构时，Remoting 因为"性能开销大、复杂、难以维护"被列入了裁剪清单。

> 💡 没有 Remoting 支持，AppDomain 就无法实现跨域对象访问——那 AppDomain 作为"隔离容器"的核心价值就没了

Stack Overflow 上 .NET 团队的人解释："You can't run code in an app domain without remoting support"（没有远程处理支持，就无法在应用程序域中运行代码）——所以 AppDomain 也被一并放到了裁剪清单上。

2. AppDomain 的资源开销大，不符合现代 .NET 的定位

.NET Core 的目标是**小巧、跨平台、高性能。**AppDomain 需要 CLR 深度支持（独立的 Loader 堆、独立的 GC 视角、跨域代理……），这些机制让运行时变得沉重。

微软的原话："AppDomains require runtime support and are generally quite expensive"（AppDomain 需要运行时支持，且通常相当昂贵）。

3. 现代架构更喜欢"进程/容器隔离"

云计算时代，隔离的最佳实践已经变成了：

- **Docker 容器**（强隔离，资源可控）
- **独立进程**（操作系统级隔离，崩溃互不影响）

相比之下，AppDomain 这种"进程内逻辑隔离"显得尴尬——隔离性不如容器/进程，灵活性又不够轻量。

三、替代品：AssemblyLoadContext（ALC）

微软给出的现代解决方案是 `AssemblyLoadContext`（.NET Core 引入，.NET Framework 中没有）。

ALC 是什么？

微软文档定义："AssemblyLoadContext 提供查找、加载和缓存托管程序集及其他依赖项的服务。为了支持动态代码的加载和卸载，它会创建隔离上下文，在自身的 AssemblyLoadContext 实例中加载代码及其依赖项。"

简单说：**ALC 是"程序集加载的上下文"**——它决定了"当代码说'我要加载 XXX.dll'时，去哪里找、加载哪个版本"。

ALC 的核心能力

每个 .NET 5+ 应用程序都隐含使用 ALC。它的关键特性：

1. 版本隔离
一个 ALC 实例对同一个程序集（按简单名称）**只能加载一个版本**。所以当你需要让不同版本的 `Newtonsoft.Json.dll` 共存时，就把它们放进**不同的 ALC**：

```
默认 ALC
└── Newtonsoft.Json v13.0.1
└── PluginA.dll（依赖 Json v13）

插件 ALC
└── Newtonsoft.Json v12.0.3
└── PluginB.dll（依赖 Json v12）
```

这是 AppDomain 时代很难优雅解决的问题——现在 ALC 天然支持。

2. 可卸载（Collectible）
创建 ALC 时可以指定 `isCollectible: true`，之后调用 `Unload()` 可以卸载整个上下文里的所有程序集：

```
var context = new AssemblyLoadContext("PluginContext", isCollectible: true);
Assembly plugin = context.LoadFromAssemblyPath("PluginB.dll");
// ... 使用插件 ...
context.Unload();  // 卸载
```

但要注意：ALC 的卸载**不像 AppDomain.Unload 那么彻底。**它依赖 GC——你必须确保所有对插件对象、类型的引用都被释放，ALC 才能在下次 GC 时被回收。这比 AppDomain 的"整体卸载"更容易内存泄漏。

ALC vs AppDomain 对比

维度	AppDomain（.NET Framework）	AssemblyLoadContext（.NET 5+）
隔离级别	进程内强隔离（独立 Loader 堆、GC 视角）	仅程序集加载隔离
跨边界调用	Remoting + 代理	不支持，需手动通过接口/反射
卸载	`AppDomain.Unload()` 同步彻底卸载	`Unload()` 异步，依赖 GC 回收
安全边界	支持 CAS 权限集	不支持，需用进程/容器
版本共存	困难	天然支持（不同 ALC 加载不同版本）
开销	大	小

> 📌 **核心差异**：AppDomain 提供的是"**进程内逻辑进程**"级别的隔离；ALC 提供的仅仅是"**程序集加载的隔离**"。前者是重武器，后者是轻武器。

四、现代 .NET 中的隔离方案全景

所以现在的局面是，微软推荐**分层隔离**：

```
层级 1：AssemblyLoadContext
    → 解决"动态加载/卸载程序集"、"多版本共存"
    → 轻量级，无安全边界

层级 2：进程隔离
    → 解决"代码崩溃互不影响"
    → 操作系统级保护

层级 3：容器隔离（Docker）
    → 解决"依赖隔离、资源限制、部署一致性"
    → 云原生时代的主流方案
```

ASP.NET Core 在 IIS 上的模型也变了：.NET Framework 时代是"一个 w3wp.exe 进程 + 多个 AppDomain 每个站点一个"；.NET Core/.NET 5+ 时代是"**每个站点一个独立进程**"（通过 `dotnet.exe` 启动），站点之间靠进程边界隔离。

五、对你的《CLR via C#》学习意味着什么？

这本书（Jeffrey Richter 著）讲的是 **.NET Framework 4.x** 时代的 CLR。所以：

✅ **书中的概念依然正确且有价值**：

- CLR 的加载机制（MSCorEE.dll → clr.dll）
- JIT 编译、GC、类型系统
- AppDomain 的 Loader 堆、跨域封送

- 这些**原理**在 .NET 5+ 的 CLR（CoreCLR）里依然存在，只是暴露给开发者的 API 变了

⚠️ **但书中的代码在 .NET 5+ 上跑不通**：

- `AppDomain.CreateDomain()` → 抛 `PlatformNotSupportedException`
- `AppDomain.Unload(domain)` → 不支持
- `MarshalByRefObject` 跨域调用 → 需改用其他机制

🎯 **实际开发时的对应方案**：

《CLR via C#》讲的	.NET 5+ 中改用
AppDomain 隔离插件	`AssemblyLoadContext`（可收集）
AppDomain 沙箱权限	进程隔离 / Docker 容器
跨 AppDomain 调用	进程间通信（gRPC、HTTP）/ 共享接口 + ALC
多版本程序集共存	多 ALC 实例

六、一个简单的 ALC 插件系统示例

感受一下现代写法：

```
using System.Runtime.Loader;

// 1. 定义一个可收集的 ALC
class PluginLoadContext : AssemblyLoadContext
{
    public PluginLoadContext() : base(isCollectible: true) { }

    // 自定义程序集解析逻辑
    protected override Assembly? Load(AssemblyName name)
    {
        // 从插件目录加载依赖
        return null;  // 交给默认逻辑
    }
}

// 2. 使用
var alc = new PluginLoadContext();
Assembly plugin = alc.LoadFromAssemblyPath(@"plugins\PluginB.dll");

// 3. 通过共享接口创建实例（主程序和插件都引用同一个接口程序集）
Type pluginType = plugin.GetType("PluginB.Plugin");
var instance = (IPlugin)Activator.CreateInstance(pluginType);
```

```
    instance.Execute();

    // 4. 卸载
    alc.Unload();
    // 注意：还要确保所有对 instance、pluginType、plugin 的引用都被释放
    // 否则 ALC 不会被 GC 回收
```

可以看到，这套写法比 AppDomain 时代**更轻量，但也更需要小心内存管理**。

🎯 总结：你需要记住的几件事

1. **AppDomain 没有被完全删除，但它的"隔离容器"语义在 .NET 5+ 中被移除**——只剩 `AppDomain.CurrentDomain` 这个壳
2. `CreateDomain()` **和** `Unload()` **在 .NET 5+ 上不可用**，微软明确说不打算再加回来
3. **替代品是** `AssemblyLoadContext`：轻量级，解决程序集动态加载/卸载和版本共存，但**不提供安全边界**
4. **强隔离用进程或 Docker 容器**——这是现代 .NET 的官方推荐
5. **《CLR via C#》第22章讲的 AppDomain 是 .NET Framework 时代的真理**，原理依然值得学，但写新项目要换工具

> 💡 一个判断标准：
>
> - 如果你维护 **.NET Framework 4.x** 老项目 → AppDomain 仍是核心机制，书里的内容直接可用
> - 如果你写 **.NET 5/6/7/8+** 新项目 → 忘掉 AppDomain，拥抱 AssemblyLoadContext + 进程/容器隔离

学《CLR via C#》的价值在于**理解 CLR 的底层的原理**——这些原理在 CoreCLR 里依然成立。但**API 层面的写法**要跟着时代走。这也是为什么 Richter 这本书虽然经典，但配合现代 .NET 文档一起看效果最好。

下一章（第23章）讲"程序集加载和反射"——这部分在 .NET 5+ 里 `AssemblyLoadContext` 就是主角，你可以直接对照着看，会很有感觉。

▾ 引用 11 篇资料作为参考

1. 🔗 应用程序域
2. 🔗 .NET 6 上的 .NET Framework 技术不可用
3. 🔗 C# AppDomain使用方法 C# AppDomain是什么以及如何使用
4. 🔗 AppDomain 类 (System)
5. 🔗 Understanding AppDomains in .NET Framework and .NET 5 to 8
6. 🔗 什么是 .NET 中的应用程序域,如何实现隔离和卸载?的全部回答
7. 🔗 關於 AssemblyLoadContext - .NET
8. 🔗 No AppDomains in .NET Core! Why?
9. 🅲 基于 AssemblyLoadContext 的.NET 插件化架构设计与实现
10. 🔗 AppDomain Class (System)
11. 🔗 About AssemblyLoadContext - .NET

（注：文档部分内容可能由 AI 生成）